

Sisteme de operare

Laborator 8

Semestrul I 2025-2026

Laborator 8

- IPC System V
 - cozi de mesaje
- POSIX threads
 - mutex-uri

System V IPC

- cozi de mesaje, semafoare, memorie partajata
- toate aceste mecanisme folosesc un set comun de functii
- in kernel, toate structurile de date corespunzatoare sunt identificate printr-un ID (intreg nenegativ)
- pentru a folosi aceste mecanisme IPC e necesar sa stim acest ID
 - de ex: ca sa trimitem/primim mesaje intr-o/dintr-o coada de mesaje avem nevoie de ID-ul cozii
- in schimb, apelurile de creare a acestor mecanisme IPC necesita o cheie (o valoare de tip *key_t*) care va fi convertita de kernel intr-un ID

IPC rendez-vous

- pt. ca doua procese sa comunice prin mecanisme System V IPC trebuie sa poata identifica aceeasi structura IPC din kernel
- in acest sens, exista 3 posibilitati

1) IPC_PRIVATE

- un proces (*server*) creeaza o noua structura IPC folosind o cheie de tip `IPC_PRIVATE`
- stocheaza ID-ul returnat de kernel undeva accesibil altui proces (*client*) care participa la IPC

ex: in fisier sau partajat prin *fork*

2) serverul si clientul specifica o cheie intr-un header comun

Pb: daca exista in kernel o alta structura care foloseste aceeasi cheie, crearea noii structuri IPC esueaza

IPC rendez-vous (cont.)

- 3) serverul si clientul folosesc o cale comuna catre un fisier si un ID de proiect
- le convertesc intr-o cheie cu ajutorul functiei *ftok*
 - pentru a accesa o structura IPC existenta se foloseste cheia cu care a fost creata
 - creare structura IPC noua
 - pt a evita folosirea unei structuri existente, apelurile de creare a mecanismelor IPC folosesc `IPC_CREAT` | `IPC_EXCL`
 - permisiuni structura IPC din kernel:
 - UID/GID proprietar
 - UID/GID creator
 - mod de acces
 - nr de secventa
 - cheie

Structuri IPC, pros & cons

- system-wide, fara reference count
 - ex: coada de mesaje cu mesaje in ea nu este stearsa din sistem cand procesul a terminat executia
 - e nevoie de apeluri sistem explicite (sau comenzi shell) pentru a o sterge
- Ex:


```
$ ipcs
```

```
$ ipcrm -{m | s | q} <id>
```

 - prin comparatie, *pipe*-urile dispar odata cu procesele care le utilizeaza
 - FIFO: ramane numele de fisier, dar datele sunt sterse
- necunoscute sistemului de fisiere
 - e nevoie de comenzi si apeluri sistem speciale pentru a lucra cu ele
- DAR, sunt reliable, au flow control, orientate pe inregistrari, pot fi procesate si in alta ordine decat FIFO

Cozi de mesaje

- create cu *msgget*

int msgget(key_t key, int flag);

ex: flag IPC_CREAT | IPC_EXCL | 0644

- attributele lor (stocate intr-o structura *msgid_ds*) pot fi manipulate cu *msgctl*

*int msgctl(int msgid, int cmd, struct msgid_ds *buf);*

Valori *cmd*:

IPC_STAT, citeste structura *msgid_ds* asociata cozii

IPC_SET, seteaza UID/GID, *mode* si dimensiunea cozii
(dimensiunea cozii poate fi crescuta doar de *root*)

IPC_RMID, sterge imediat coada din sistem (si mesajele din ea; alte procese care folosesc ulterior coada primesc EIDRM)

Obs: pt SET si RMID e nevoie fie de drepturi de root fie ca

UID creator = UID proprietar

Cozi de mesaje (cont.)

- mesajele se scriu/citesc in/din coada cu *msgsnd/msgrcv*
- apelurile folosesc o structura definita de utilizator

```
struct msg {  
  
    long mtype;                // tip mesaj, valoare pozitiva  
  
    char mtext[BUFSIZE];      // date mesaj  
  
}  
  
int msgsnd(int msgid, const void *ptr, size_t nbytes, int flag);  
  
int msgrcv(int msgid, void *ptr, size_t nbytes, long type, int flag);
```

- pt. type = 0 la *msgrcv* se citeste primul mesaj din coada (politica FIFO), altfel se citeste primul mesaj din coada cu acel tip

POSIX threads

- creare

```
int pthread_create(pthread_t pth, const pthread_attr_t *attr,  
void *(*start_routine)(void *), void *arg);
```

- porneste un nou thread in proces apeland *start_routine* cu *arg* ca parametru
- *attr* setat cu *pthread_attr_init*, NULL indica attributele implicite

- terminare

- normal, prin iesirea din *start_routine*
- cu *pthread_exit*
- cu *pthread_cancel*
- *exit* din orice thread al procesului
- iesire din *main*

POSIX threads (cont.)

- tipuri de thread-uri (atribut)
 - detached
 - la terminarea thread-ului resursele sale sunt eliberate fara sa fie nevoie ca alt thread sa apeleze *pthread_join*, INDIFERENT de cauza terminarii sale
 - se face cu *pthread_detach*
int pthread_detach(pthread_t thread);
 - joinable
 - eliberarea resurselor thread-ului se face doar cand un alt thread cheama *pthread_join*

- *pthread_exit*

*void pthread_exit(void *retval);*

- termina thread-ul apelant
- paseaza *retval* catre *pthread_join*
- resursele partajate ale procesului NU sunt eliberate !

POSIX threads (cont.)

- pthread_join

*int pthread_join(pthread_t thread, void **retval);*

- asteapta terminarea thread-ului *thread*
- daca *thread* a terminat deja, apelul e neblocant
- in *retval* se regaseste starea de terminare a thread-ului *thread*
- daca *thread* a fost anulat, **retval* = PTHREAD_CANCELED
- *thread* trebuie sa fie joinable

POSIX threads (cont.)

- `pthread_cancel`

int pthread_cancel(pthread_t thread);

- trimite o cerere de anulare a executiei lui *thread*
- daca posibilitatea de anulare e dezactivata, se asteapta pana la activarea ei
- altfel, anularea poate fi
 - *asincrona* – *thread* poate fi terminat imediat (dar nu e garantat)
 - *intarziata* – pana cand *thread* ajunge intr-un *cancellation point*
- anularea determina apelul handlerelor de clean-up + destructori de date specifice *thread* + terminare *thread* (`pthread_exit`)

Obs: pthread_cancel doar trimite o cerere de anulare intr-o coada, nu asteapta anularea efectiva si consecintele ei

Ex: producer-consumator POSIX

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define ITEMS      8
5 #define BUFFER_SIZE 4
6 int buffer[BUFFER_SIZE];
7
8 void *producer(void*);
9 void *consumer(void*);
10
11 int spaces, items, tail;
12 pthread_cond_t space, item;
13 pthread_mutex_t buffer_mutex;
14
15 int main()
16 {
17     pthread_t producer_thread, consumer_thread;
18     void *thread_return;
19     int result;
20
21     spaces = BUFFER_SIZE;
22     items = 0;
23     tail = 0;
24
25     pthread_mutex_init(&buffer_mutex, NULL);
26     pthread_cond_init(&space, NULL);
27     pthread_cond_init(&item, NULL);
28
29     if(pthread_create(&producer_thread, NULL, producer, NULL) ||
30        pthread_create(&consumer_thread, NULL, consumer, NULL))
31         exit(1);
32
33     if(pthread_join(producer_thread, &thread_return))
34         exit(1);
35     else
36         printf("producer returns with %d\n", (int)thread_return);
37
38     if(pthread_join(consumer_thread, &thread_return))
39         exit(1);
40     else
41         printf("consumer returns with %d\n", (int)thread_return);
42     exit(0);
43 }
44
```

POSIX Mutex Locks

- crearea si initializarea lock-ului mutex

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex, NULL);
```

- obtinerea si eliberarea lock-ului

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

Sisteme de operare

Laborator 8

POSIX threads, System V IPC

1. Creati un nou subdirector *lab8/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab8/bin*.

2. Rescrieti programul C **race.c** din laboratorul 4 in care doua procese, parinte si copil, scriu concurent propriul string pe ecran, scrierea fiind facuta caracter cu caracter. In loc de procese, folositi doua thread-uri POSIX create in thread-ul *main*. Fiecare thread apeleaza functia *myprint* de 10 ori intr-o bucla pentru a evidentia mai clar race condition-ul. Folositi un *mutex* POSIX pentru a impiedica producerea race condition-ului si a obtine un output neintretesut pe ecran.

3. Scrieti doua programe C **msg-echod.c** si **msg-echo.c** care implementeaza un serviciu de *echo* la fel ca in laboratorul 7, de data aceasta folosind cozi de mesaje System V IPC in loc de FIFOs. Practic, in loc sa foloseasca un fisier *FIFO*, programul **msg-echod.c** creeaza o coada de mesaje cu ajutorul *msgget* (parametrul *key* il generati cu functia *ftok*). Apoi apeleaza in bucla functia *msgrcv* pentru a primi mesaje de la clienti pe care le scrie inapoi in coada de mesaje (serviciu de *echo*) folosind *msgsnd*. Programul **msg-echod.c** se lanseaza in background ca serviciu de echo dupa cum urmeaza:

```
$ gcc -o msg-echod msg-echod.c
$ msg-echod keyfile &          # keyfile e un fisier pe care il creati voi
$ ipcs -q                      # vizualizati coada creata
```

Programul **msg-echo.c** foloseste parametrul primit in linie de comanda (*keyfile*) ca argument pt functia *ftok* si astfel poate identifica si obtine cu ajutorul *msgget* un ID pentru coada de mesaje creata de *echod*. Apoi, citeste in bucla caractere de la tastatura folosind functia *fgets* si asambleaza mesaje cu tipul dat de PID-ul sau (obtinut cu *getpid*) pe care le trimite in coada de mesaje cu ajutorul functiei *msgsnd*. Apoi citeste din coada de mesaje cu ajutorul *msgrcv* ecoul caracterelor trimise catre serviciul *echod* pe care, odata primite, le tipareste pe ecran. Programul se termina cand utilizatorul apasa *Ctrl-c*.

```
$ gcc -o msg-echo msg-echo.c
$ msg-echo keyfile
```

La final, nu uitati sa faceti curatenie:

```
$ jobs
$ kill %<n>                # n e numarul de job al msg-echod afisat
                          # de comanda jobs de mai sus

$ ipcs -q
$ ipcrm -q <id>           # id e id-ul cozii de mesaje tiparit de comanda
                          # ipcs de mai sus
```

Pentru a evita ultimele doua comenzi de mai sus, in programul **msg-echod.c** puteti sa prindeti semnalul *SIGTERM* si sa stergeti coada de mesaje din program atunci cand se executa comanda *kill* de mai sus.

4. Modificati programul C **msg-echod.c** de mai sus pentru a oferi un serviciu **multi-threaded** de *echo* folosind cozi de mesaje System V IPC. Noul program, **msg-mt-echod.c**, creeaza o coada de mesaje cu ajutorul *msgget* (parametrul *key* il generati cu functia *ftok*). Apoi apeleaza in bucla functia *msgrcv* pentru a primi mesaje de la clienti, Spre deosebire de **msg-echod.c**, care este *single-threaded*, **msg-mt-echod.c** creeaza un thread POSIX separat pentru fiecare nou mesaj primit. Serverul copiaza mesajul primit cu *msgrcv* si il paseaza noului thread creat care il scrie inapoi in coada de mesaje (serviciu de *echo*) folosind *msgsnd*. Programul **msg-mt-echod.c** se lanseaza in background ca serviciu de *echo* dupa cum urmeaza:

```
$ gcc -o msg-mt-echod msg-mt-echod.c -lpthread
$ msg-mt-echod keyfile &          # keyfile e un fisier pe care il creati voi
$ ipcs -q                          # vizualizati coada creata
```

Pentru a testa functionalitatea serverului folositi programul client **msg-echo.c** dezvoltat in exercitiul anterior. Ca sa testati functionalitatea serverului multithreaded lansati mai multe programe client in mod concurent (de pilda folosind un shell script simplu care lanseaza in background un numar de clienti).

Cand sunt mai multi clienti activi simultan, cum izolati traficul unui client cu serverul de traficul altor clienti? Mai exact, cum faceti ca raspunsul serverului sa ajunga exact la clientul caruia trebuie sa-i raspunda, si nu altui client pe care-l deserveste de asemenea?

La final, ca la exercitiul anterior, nu uitati sa faceti curatenie in sistem stergand coada de mesaje fie prin program, fie din shell.

5. Modificati programul C **msg-mt-echod.c** de mai sus a.i. sa emuleze comportamentul unui server de remote shell, **rshd.c**. Cand serverul **rshd** primeste un string de la un client **msg-echo** il interpreteaza ca pe o comanda shell, o executa si intoarce rezultatul executiei catre client. Mai jos aveti un exemplu de executie al serverului **rshd**:

```
$ gcc -o rshd rshd.c -lpthread
$ rshd keyfile &          # keyfile e un fisier pe care il creati voi
$ ipcs -q                  # vizualizati coada creata
```

in vreme ce clientul trebuie apelat cu mesaje care au sens, i.e. string-uri care reprezinta de fapt comenzi:

```
$ msg-echo keyfile
ls -l
...
ps
...
Ctrl-c
$
```

unde punctele de suspensie de mai sus inlocuiesc rezultatul executiei comenzii trimis inapoi de catre **rshd**.

Indicatie: in thread-ul de serviciu, *rshd* apeleaza secventa *vfork/exec/wait* pentru a executa comanda primita de la client. Pentru a recupera output-ul comenzii executate de *exec* si a-l putea trimite inapoi catre client folosind coada de mesaje, puteti folosi un *pipe* redirectat corespunzator. Pentru a

nu mai parsa string-ul care reprezinta comanda, folositi in *exec* comanda “*sh -c*” care executa o comanda care urmeaza dupa flagul *-c*.

6. Modificati programul C **rshd.c** a.i. sa emuleze comportamentul OOB (Out-Of-Band data) prin care serverele de retea sunt anuntate prioritar de sosirea unui caracter (sau secventa de caractere) de control, cum ar fi de pilda *Ctrl-c* intr-o sesiune de *ssh* pentru a termina o comanda aflata in executie la distanta. Noua versiune a programului, **rshd-oob.c**, primeste caracterul de control, *Ctrl-c* in exemplu nostru, il interpreteaza (*Ctrl-c* inseamna terminarea comenzii) si executa dispozitia asociata secventei de control (termina comanda).

N.B. Cand programul client, **msg-echo**, primeste *Ctrl-c* de la tastatura se va termina abrupt. Nu aceasta este functionalitatea pe care trebuie sa o implementati, deci trebuie sa modificati si codul lui **msg-echo.c** pentru a evita terminarea programului client si a fi capabil sa execute o secventa de felul urmator:

```
$ msg-echo keyfile
sleep 300
Ctrl-c
ps
...
ls -l
...
$
```

In exemplul de mai sus, comanda *sleep 300* este folosita pentru a emula executia unei comenzi care dureaza mult.